Experiment 10: Autoencoder for Image Denoising

1. Dataset Source

Dataset used: MNIST dataset
Source link: https://keras.io/api/datasets/mnist/

2. Dataset Description

- The MNIST dataset consists of grayscale images of handwritten digits (0–9).
- It is commonly used for image processing and deep learning tasks.

Features

- Each image is 28 × 28 pixels (grayscale)
- Pixel values range from 0 to 255

Target Variable

- The target is the original clean image itself
- Type: Reconstruction problem (unsupervised learning)

Dataset Size

- Training samples: 60,000 images
- Testing samples: 10,000 images

Key Characteristics

- Simple and clean dataset
- Suitable for reconstruction and denoising tasks
- Noise can be artificially added to simulate real-world conditions
- Does not require labeled outputs

3. Mathematical Formulation of the Algorithm

An Autoencoder consists of two main components: encoder and decoder.

Encoder

$h = f(W_1x + b_1)$

Latent Representation

$z = f(W_2h + b_2)$

Decoder

$\hat{x} = f(W_3z + b_3)$

Activation Functions

ReLU:
$f(x) = \max(0, x)$

Sigmoid:
$f(x) = 1 / (1 + e^{(-x)})$

Loss Function (Binary Crossentropy)

$L = -\Sigma [x \log(\hat{x}) + (1 - x) \log(1 - \hat{x})]$

Alternative Loss (Mean Squared Error)

$L = (1/n) \Sigma (x - \hat{x})^2$

4. Algorithm Limitations

- Performance depends on quality and type of noise added
- May fail to generalize to unseen noise patterns
- Can produce blurry reconstructions
- Requires sufficient training data
- Not suitable for highly complex images without deeper architectures

5. Methodology / Workflow

Steps Involved

1. Import required libraries

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

2. Load dataset

```
(x_train, _), (x_test, _) = keras.datasets.mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
```

3. Normalize data

```
x_train = x_train / 255.0
x_test = x_test / 255.0
```

4. Add noise to images

```
noise_factor = 0.5

x_train_noisy = x_train + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_train.shape)
x_test_noisy = x_test + noise_factor * np.random.normal(loc=0.0, scale=1.0, size=x_test.shape)

# Clip values to stay in range
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

5. Build autoencoder model

```
model = keras.Sequential([

    # Encoder
    keras.layers.Flatten(input_shape=(28,28)),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(64, activation='relu'),

    # Decoder
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(784, activation='sigmoid'),
    keras.layers.Reshape((28,28))
])
```

6. Compile model

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy'
)
```

7. Train model

```
history = model.fit(
    x_train_noisy, x_train,
    epochs=10,
    batch_size=128,
    validation_data=(x_test_noisy, x_test)
)
```

```
Epoch 1/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 9s 16ms/step - loss: 0.2078 - val_loss: 0.1537
Epoch 2/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 11s 24ms/step - loss: 0.1446 - val_loss: 0.1358
Epoch 3/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - loss: 0.1332 - val_loss: 0.1285
Epoch 4/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 11s 15ms/step - loss: 0.1271 - val_loss: 0.1241
Epoch 5/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - loss: 0.1235 - val_loss: 0.1210
Epoch 6/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 15ms/step - loss: 0.1210 - val_loss: 0.1191
Epoch 7/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 10s 15ms/step - loss: 0.1189 - val_loss: 0.1179
Epoch 8/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - loss: 0.1171 - val_loss: 0.1168
Epoch 9/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 7s 16ms/step - loss: 0.1157 - val_loss: 0.1153
Epoch 10/10
469/469 ━━━━━━━━━━━━━━━━━━━━ 6s 12ms/step - loss: 0.1144 - val_loss: 0.1140
```

8. Denoise images using trained model

```
decoded_images = model.predict(x_test_noisy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step
```

9. Display and compare results

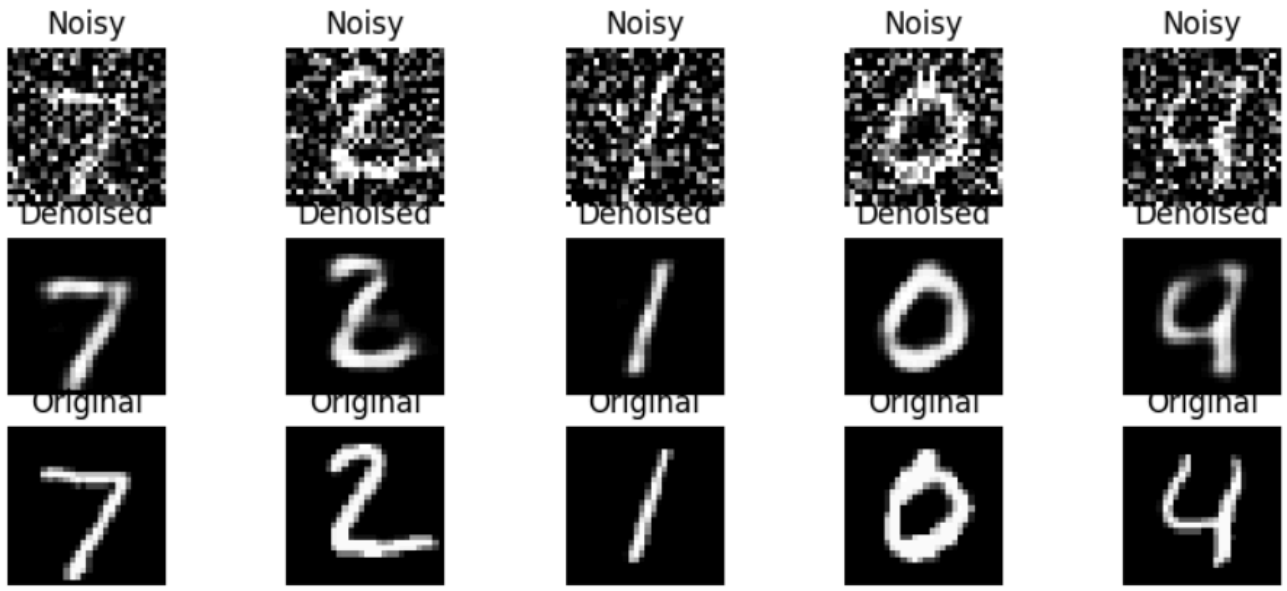
```
n = 5
plt.figure(figsize=(10,4))

for i in range(n):
    # Noisy image
    plt.subplot(3, n, i+1)
    plt.imshow(x_test_noisy[i], cmap='gray')
    plt.title("Noisy")
    plt.axis('off')

    # Denoised image
    plt.subplot(3, n, i+n+1)
    plt.imshow(decoded_images[i], cmap='gray')
    plt.title("Denoised")
    plt.axis('off')

    # Original image
    plt.subplot(3, n, i+2*n+1)
    plt.imshow(x_test[i], cmap='gray')
    plt.title("Original")
    plt.axis('off')

plt.show()
```



Workflow Summary

Noisy Image → Encoder → Latent Representation → Decoder → Reconstructed Image

6. Performance Analysis

- Model successfully reconstructs clean images from noisy inputs
- Loss decreases over epochs indicating learning

Interpretation

- Denoised images closely resemble original images
- Some minor blurring may be observed
- Model captures important features while removing noise

7. Hyperparameter Tuning

Parameters Tuned

- Number of hidden layers
- Number of neurons in encoder/decoder
- Noise factor
- Batch size
- Number of epochs

Observations

- Moderate noise levels (around 0.5) give best learning
- Too much noise reduces reconstruction quality
- Increasing neurons improves reconstruction up to a limit
- Optimal epochs around 10

Impact

- Improved denoising quality
- Reduced reconstruction loss
- Better visual output

Conclusion

The autoencoder model successfully learned to remove noise from images and reconstruct clean outputs. The experiment demonstrated the effectiveness of unsupervised learning techniques in feature extraction and image denoising tasks, while also highlighting limitations such as blurring and sensitivity to noise levels.